# РОЗДІЛ 1

# АНАЛІЗ ЗАДАЧІ TEXT-TO-SQL, НАБОРІВ ДАНИХ І МЕТОДІВ АДАПТАЦІЇ МОВНИХ МОДЕЛЕЙ

У розділі зібрано постановку задачі перетворення природномовного запиту на SQL, метрики, якими порівнюють розв'язки, еталонні й навчальні набори, методи адаптації мовних моделей і наявні засоби контролю якості анотацій. Огляд спирається на першоджерела наборів і методів, на рецензовані огляди застосування великих мовних моделей до Text-to-SQL [1] і низькорангової адаптації [2], а також на препринт огляду етапів LLM-систем Text-to-SQL [3]. Власні методи й числа аудитів у цей розділ не входять. Прогалини, з яких виходять наступні розділи, сформульовано в кінці пунктів 1.4.1–1.4.3 і у висновках.

## 1.1 Задача Text-to-SQL та розвиток підходів до її розв'язання

Для побудови SQL за питанням користувача потрібно врахувати зміст питання та структуру бази даних. Перед розглядом методів розв'язання цієї задачі визначимо її вхідні дані, очікуваний результат і вимоги до правильності запиту.

### 1.1.1 Постановка та формалізація задачі Text-to-SQL

Зростання обсягів даних у базах даних і аналітичних сховищах підвищує потребу в засобах їх пошуку та аналізу. Водночас отримання потрібної вибірки зазвичай потребує володіння мовою SQL, що обмежує доступ до даних для користувачів без відповідної технічної підготовки. Подолання цього бар'єра зумовило розвиток засобів, які дають змогу формулювати інформаційну потребу природною мовою [3].

Задача Text-to-SQL полягає в автоматичному перетворенні питання природною мовою на SQL-запит до заданої реляційної бази даних. Виконання цього запиту системою керування базами даних (СКБД) має надати відповідь на поставлене питання. Самої синтаксичної коректності запиту недостатньо. Він має звертатися до потрібних таблиць і колонок. Умови відбору, з'єднання, агрегування та інші операції мають відповідати змісту

питання [4, 5].

У термінах оброблення природної мови Text-to-SQL належить до семантичного парсингу [4], оскільки питання природною мовою відображають у формальне представлення його змісту. У цій задачі таким представленням є SQL-запит. Далі розглядається саме побудова такого запиту. Повний природномовний інтерфейс до бази даних є ширшим і може додатково підтримувати діалогове уточнення питання та подання результату користувачеві [6].

Позначимо питання природною мовою через $q$, а схему цільової бази даних через $S$. Схема містить назви таблиць і колонок, типи даних, первинні та зовнішні ключі й інші оголошені обмеження. Через $D$ позначимо поточний стан бази, тобто вміст її таблиць. За незмінної схеми цей вміст може змінюватися. Відповідно, той самий запит може повертати різні результати.

У базовій постановці вхідними даними є питання $q$ та схема $S$ [4]. Для інтерпретації питання можуть знадобитися й додаткові відомості. До них належать описи колонок, значення з бази, предметні знання та правила обчислення показників [5]. Позначимо ці відомості через $E$ (evidence). За їх відсутності $E = \varnothing$.

З урахуванням введених позначень запишемо перетворення Text-to-SQL:

$$(q, S, E) \xrightarrow{\text{Text-to-SQL}} y, \tag{1.1}$$

де $y$ — SQL-запит у заданому діалекті $\delta$. Потрібно побудувати запит, синтаксично коректний у цьому діалекті та узгоджений зі схемою $S$. Його зміст має відповідати питанню $q$ з урахуванням $E$.

Для отримання відповіді SQL-запит $y$ виконують у СКБД на поточному стані бази $D$. Результат виконання позначимо через

$$R = \text{Exec}_\delta(y, D). \tag{1.2}$$

Оператор $\text{Exec}_\delta$ враховує правила заданого діалекту $\delta$. Результат $R$ визначений за умови успішного виконання. Помилку виконання слід відрізняти від порожнього результату. Наприклад, коректний запит на відбір рядків поверне порожній результат, якщо жодний рядок не задовольняє заданої умови.

Розглянемо питання «Як звати співаків, старших за 30 років?» (рис. 1.1). Таблиця `singer` містить колонку `name` з іменами та колонку `age` з віком у

роках. Слово «співаків» зіставляється з таблицею `singer`, а «як звати» — з колонкою `name`. Фрагмент «старших за 30 років» задає умову `age > 30`. Виконання запиту повертає імена співаків, які задовольняють цю умову. Записи з віком рівно 30 років до результату не входять.

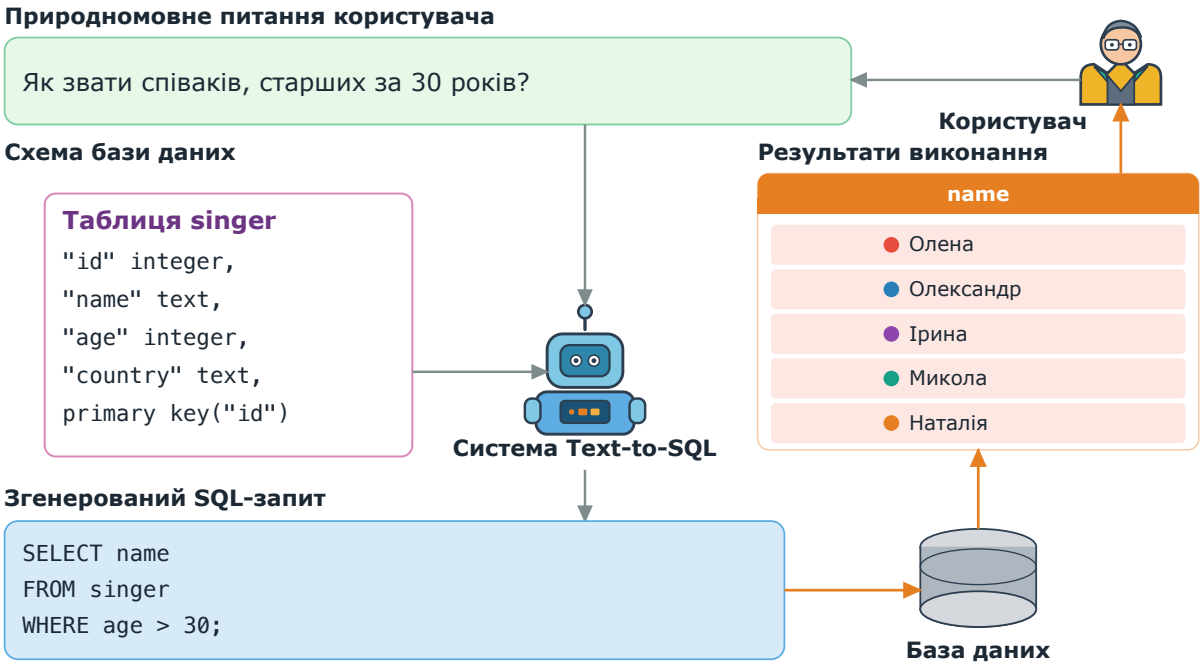



**Рисунок 1.1** – Загальна схема природномовного доступу до бази даних із використанням систем Text-to-SQL

Семантична правильність запиту визначається його відповідністю змісту питання в контексті заданої бази. Різні за структурою SQL-запити можуть бути семантично еквівалентними. Водночас збіг результатів на одному стані $D$ не доводить їхньої еквівалентності на інших станах. Наприклад, заміна `age > 30` на `age >= 30` у наведеному запиті змінює зміст умови. Проте обидва запити дадуть однакову відповідь, якщо в таблиці немає співаків віком рівно 30 років. Отже, слід розрізняти структурний збіг SQL, збіг результатів виконання та відповідність запиту питанню [1].

У наборах даних питанню $q$ відповідає еталонний SQL-запит $y^*$. Пару $(q, y^*)$ використовують для навчання та оцінювання систем. При цьому статус еталонного не гарантує правильності запиту.

Основні труднощі Text-to-SQL пов'язані з неоднозначністю природної мови, зіставленням понять питання з таблицями та колонками бази даних і визначенням потрібних SQL-операцій. Складні питання можуть вимагати з'єднання таблиць, агрегування та вкладених запитів. Окремою проблемою

є узагальнення на нові схеми й предметні області, не представлені в навчальних даних.

### 1.1.2 Еволюція підходів до Text-to-SQL

З погляду розвитку мовних моделей підходи до Text-to-SQL умовно поділяють на методи на основі правил, нейронні моделі, попередньо навчені мовні моделі (PLM) та великі мовні моделі (LLM) [7]. Межі між цими групами не є чіткими, оскільки нові системи продовжують використовувати структурні обмеження й інші компоненти попередніх підходів. Водночас великі мовні моделі є масштабованим різновидом попередньо навчених моделей. Узагальнення розвитку підходів наведено на рис. 1.2.

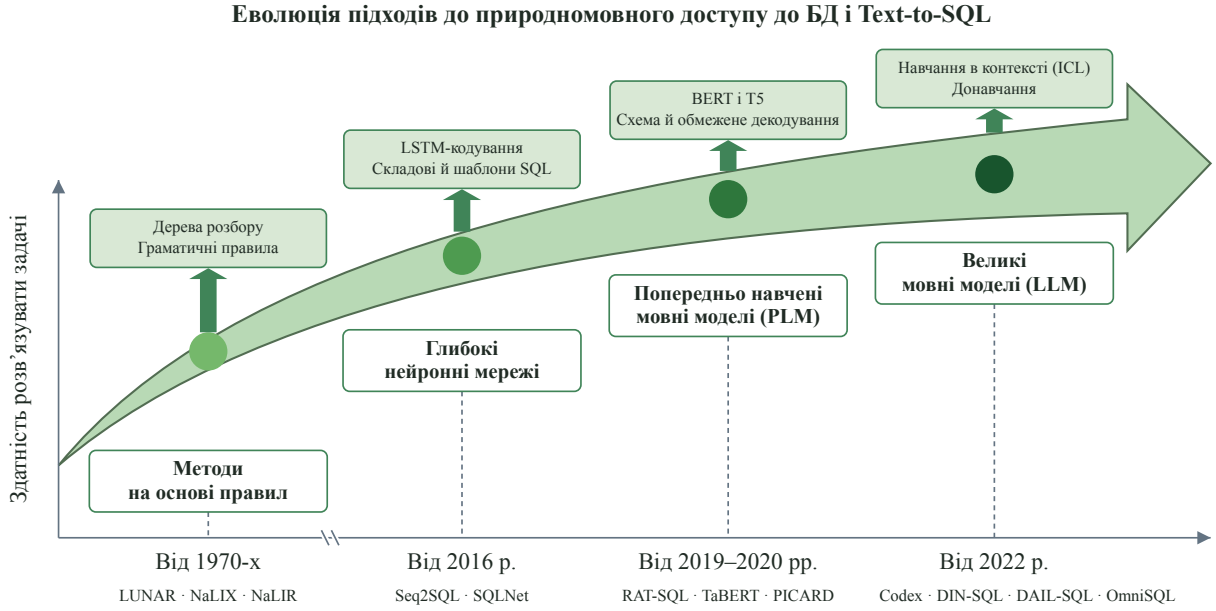



**Рисунок 1.2** – Еволюція підходів до природномовного доступу до баз даних і Text-to-SQL

**Методи на основі правил і шаблонів.** Типовий процес у системах на основі правил передбачав лексичний і синтаксичний аналіз питання. Синтаксичну структуру питання часто подавали як дерево синтаксичного розбору. Семантичні правила зіставляли елементи цього дерева з таблицями, колонками, значеннями та операціями над даними. За результатами зіставлення система формувала запит за визначеними правилами й шаблонами. Конкретний формат залежав від системи. LUNAR будувала процедуру отримання відповіді [8], NaLIX — XQuery до XML-даних [9], а NaLIR — SQL-запит до реляційної бази [10].

Оскільки правила та відповідності задавали вручну, підтримка нових

формулювань, елементів схеми й конструкцій запиту потребувала змін у системі. Це ускладнювало її супровід і перенесення на інші предметні області.

**Глибокі нейронні мережі.** Із середини 2010-х років для семантичного парсингу почали застосовувати архітектури «кодувальник–декодувальник» на основі рекурентних нейронних мереж, зокрема LSTM. Dong і Lapata застосували LSTM із механізмом уваги для породження логічних форм, хоча їхня робота не була безпосередньо системою Text-to-SQL [11]. На відміну від систем на основі правил, такі моделі навчали встановлювати відповідності між природномовним питанням і формальним запитом на основі навчальних прикладів. У Text-to-SQL такий приклад пов'язував питання і схему бази даних з еталонним SQL-запитом. Seq2SQL [12] обирала агрегатну функцію й колонку для `SELECT`, а потім генерувала умови `WHERE`. Оскільки однакові умови можна записати в різному порядку, генератор `WHERE` навчали з підкріпленням і оцінювали не дослівний збіг з еталоном, а результат виконання запиту. SQLNet [13], у свою чергу, розв'язувала ту саму проблему без навчання з підкріпленням. Вона заповнювала складові фіксованого шаблону SQL і передбачала колонки умов як невпорядковану множину із механізмом уваги до колонок.

**Попередньо навчені мовні моделі.** Наприкінці 2010-х років у Text-to-SQL почали використовувати попередньо навчені трансформерні моделі. Ключовою відмінністю від попереднього етапу стало попереднє навчання моделі на великих текстових корпусах із подальшою адаптацією до Text-to-SQL. Під час адаптації модель донавчали на прикладах, що пов'язували питання і схему бази даних з еталонним SQL-запитом. RAT-SQL кодувала зв'язки між елементами схеми та відповідності між словами питання й елементами схеми за допомогою механізму самоуваги з урахуванням відношень. У варіанті RAT-SQL + BERT початкові представлення слів питання й назв елементів схеми отримували за допомогою BERT [14]. TaBERT створили шляхом продовження попереднього навчання BERT на таблицях і пов'язаних із ними текстах. Потім TaBERT використовували як кодувальник у семантичному парсері та донавчали разом із ним на прикладах Text-to-SQL [15]. У PICARD під час декодування донавченої T5 інкрементальний синтаксичний аналіз відхиляв токени, які не могли бути допустимим продовженням SQL-запиту [16].

**Великі мовні моделі.** Перехід до великих мовних моделей був

пов'язаний із масштабуванням попередньо навчених моделей за кількістю параметрів, обсягом навчальних даних і обчислювальними витратами. Довше контекстне вікно давало змогу одночасно подавати моделі інструкцію, опис схеми бази даних та, за потреби, приклади «питання — SQL». У 2022 році Rajkumar та ін. показали можливість використання Codex для генерації SQL у такій постановці без спеціалізованого донавчання [17]. DIN-SQL розкладає задачу на зв'язування зі схемою, декомпозицію, генерацію SQL і самокорекцію [18]. DAIL-SQL досліджує добір і подання прикладів для навчання в контексті [19]. Водночас донавчання зберегло своє значення. Великі мовні моделі донавчають повністю або оновлюють лише невелику частину параметрів, наприклад низькорангові адаптери LoRA [7, 20]. Зокрема, моделі OmniSQL донавчено на синтетичному наборі SynSQL-2.5M разом із навчальними частинами Spider і BIRD, а цільова відповідь містить покрокове пояснення та фінальний SQL-запит [21].

### 1.1.3 Метрики exact match і execution accuracy

Еталони систем Text-to-SQL використовують метрики, які охоплюють і правильність SQL-запиту, і його ефективність. Цими метриками перевіряють, що система дає точний SQL і що запит виконується ефективно на справжніх базах. Метрики поділяються на дві групи: зіставлення вмісту і оцінювання за результатом виконання [3]. Рисунок 1.3 показує чотири метрики цієї класифікації на одному питанні і одному еталонному запиті.

Нехай $N$ — кількість прикладів, $Y_n$ — еталонний запит $n$-го прикладу, а $\hat{Y}_n$ — згенерований. $V_n$ і $\hat{V}_n$ — результати виконання цих запитів на тій самій базі. Індикатор $\mathbf{1}(\cdot)$ дорівнює 1, якщо умова в дужках виконана, і 0 інакше.

Зіставлення вмісту дивиться, наскільки структура згенерованого запиту збігається зі структурою еталонного. Таке оцінювання потрібне, щоб перевірити, чи модель дотримується правильного синтаксису і структури SQL, навіть якщо запит не завжди є найекономнішим [3].

Зіставлення компонент (component matching, CM) оцінює кожен компонент запиту окремо, зокрема `SELECT`, `FROM` і `WHERE` [3, 4]. Нехай $A(Y_n^C)$ — множина складників компоненти $C$ еталонного запиту, а $A(\hat{Y}_n^C)$ — множина складників тієї самої компоненти згенерованого. Порядок складників до

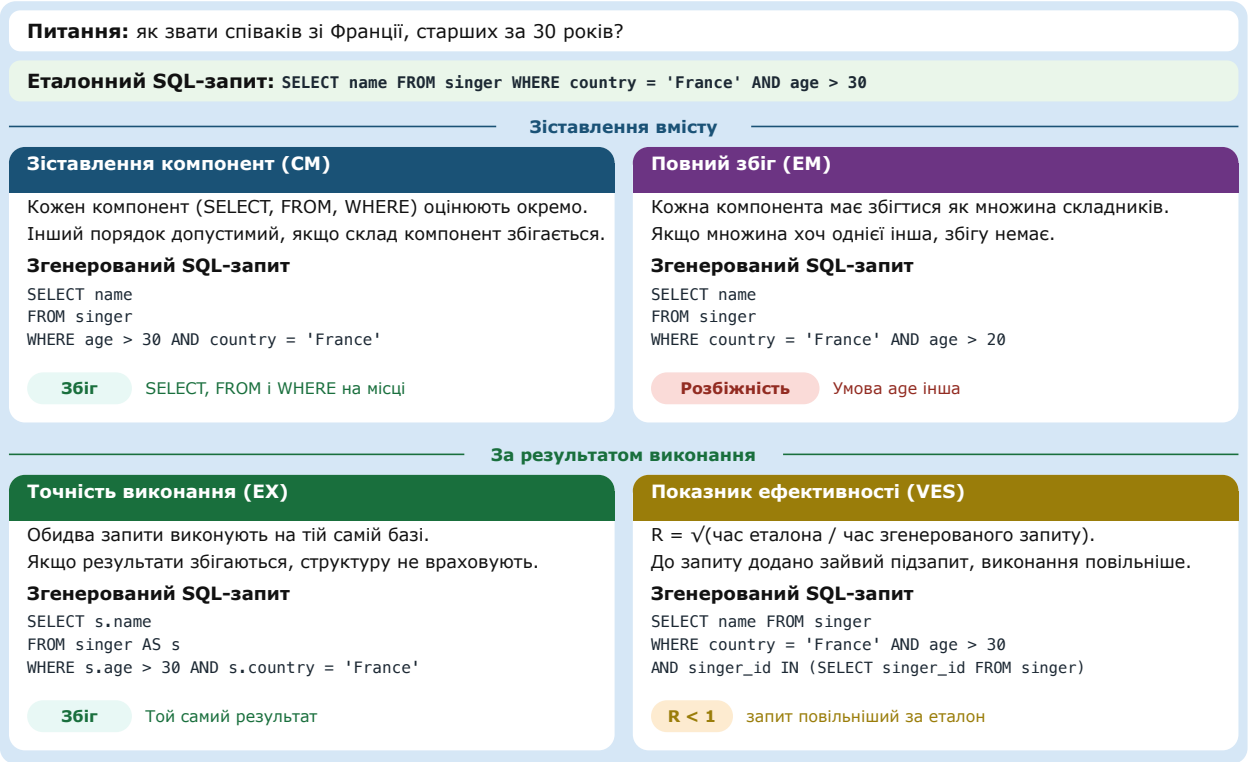



**Рисунок 1.3** – Чотири метрики оцінювання за оглядом Mohammadjafari et al. Зіставлення компонент і повний збіг дивляться на структуру запиту. Точність виконання і показник ефективності дивляться на результат і на час.

множини не входить:

$$CM^C = \frac{1}{N}\sum_{n=1}^{N} \mathbf{1}\big(A(Y_n^C) = A(\hat{Y}_n^C)\big). \tag{1.3}$$

На рисунку для `WHERE` складники ті самі: `country = 'France'` і `age > 30`, тож індикатор дорівнює 1. Для `ORDER BY` і `HAVING` порядок складників зберігають; рисунок цих компонент не показує.

Повний збіг (exact matching, EM) вимагає того самого збігу від кожної компоненти. Нехай $\mathcal{C}$ — компоненти запиту. Тоді

$$EM = \frac{1}{N}\sum_{n=1}^{N} \mathbf{1}\Big(\bigwedge_{C\in\mathcal{C}} \big(A(Y_n^C) = A(\hat{Y}_n^C)\big)\Big). \tag{1.4}$$

Індикатор дорівнює 1 лише тоді, коли рівність множин виконана для кожної $C \in \mathcal{C}$. Переставлення умов усередині однієї компоненти повного збігу не ламає [4]. Зв'язка до множини `WHERE` не входить: це окрема компонента, $\{$`AND`$\}$ і $\{$`OR`$\}$ не рівні, тож заміна однієї на іншу повного збігу не дає. Підзапит усередині `WHERE` є одним складником цієї множини і не дописується до зовнішніх

`SELECT` і `FROM`. На рисунку в згенерованому запиті стоїть `age > 20` замість `age > 30`, тож множина `WHERE` інша і повного збігу немає. Запит, правильний за результатом, усе одно не зараховують, якщо множина складників хоч однієї компоненти інша [3].

Оцінювання за результатом виконання дивиться на фактичну роботу згенерованого запиту, коли його виконують на базі. Ці метрики перевіряють, що запит дотримується правильної структури, повертає правильний результат і виконується ефективно в реальних умовах [3].

Точність виконання (execution accuracy, EX) перевіряє, чи згенерований запит, виконаний на базі, повертає той самий результат, що й еталонний [3, 4]:

$$EX = \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}(V_n = \hat{V}_n). \tag{1.5}$$

Метрика дивиться на правильність результату незалежно від структури запиту. Для прикладу зі співаками інший текст із тим самим результатом зберігає точність виконання. Збіг порожніх результатів можливий і тоді, коли обидва запити не відповідають питанню; метрика залежить від вмісту бази [1, 22].

Показник ефективності (valid efficiency score, VES) вимірює обчислювальну ефективність згенерованого запиту порівняно з еталонним. Запит може повернути правильний результат і водночас бути неефективним, бо потребує зайвих обчислень. Метрика карає додаткову складність, зокрема зайвий підзапит або зайве з'єднання, навіть за збігу результатів [3, 5]. На $N$ прикладах VES оцінює і правильність, і ефективність кожного згенерованого запиту:

$$\text{VES} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}(V_n = \hat{V}_n) \cdot R(Y_n, \hat{Y}_n), \tag{1.6}$$

де відносна ефективність

$$R(Y_n, \hat{Y}_n) = \sqrt{\frac{E(Y_n)}{E(\hat{Y}_n)}} \tag{1.7}$$

є коренем із відношення часу виконання еталонного запиту до часу згенерованого. Підсумковий VES є середнім цих величин за всіма $N$ прикладами. Оцінка 1 означає водночас точність і оптимальну ефективність [3]. На вели-

ких базах BIRD головними метриками є EX і VES, бо одну відповідь записують різними коректними запитами [5].

Порівнянними числа повного збігу й точності виконання різних систем є тоді, коли вони пораховані на тих самих золотих парах і на тому самому авторському розбитті. Це розбиття задають набори, розглянуті далі.

## 1.2 Еталонні та навчальні набори даних

Еталонний набір і навчальний виконують різні ролі. Еталон (WikiSQL, Spider, BIRD) має авторське розбиття і золоті пари, тож exact match і execution accuracy різних систем ставлять поряд. Навчальним є train-частина Spider чи BIRD або синтетичний корпус, на якому модель донавчають. Автоматичний поділ рядків набору, викладеного на Hugging Face, дає тестову частку, але не еталон: офіційні train і dev Spider розведені за базами, і статті звітують саме на цьому розбитті [4, 22]. Випадкова частка рядків, особливо коли одна схема потрапляє в обидві частини, дає непорівнянне число.

### 1.2.1 WikiSQL і Spider

WikiSQL є одним із перших великих наборів Text-to-SQL: 80 654 пари «питання — SQL» на 24 241 таблиці Вікіпедії [12]. Запит стосується однієї таблиці. З'єднання, вкладені підзапити і складна агрегація в постановку не входять. Оцінювання ведуть за збігом логічної форми і за точністю виконання.

Spider знімає це обмеження і фіксує міждоменне оцінювання [4]. У наборі 10 181 питання і 5 693 унікальні SQL-запити на 200 базах у 138 доменах. Схеми містять кілька таблиць і зовнішні ключі. Запити покривають вкладеність, операції над множинами і групування з агрегацією. Питання писали студенти, які володіють SQL; пари проходили окремий перегляд SQL, перегляд питання і фінальний перегляд. Train і dev розведені за базами: схеми dev у навчанні відсутні. Через це Spider лишається опорним еталоном міждоменного Text-to-SQL [1, 19].

Поділ за питаннями, а не за базами, завищує оцінку узагальнення на нові запити [22]. Тому dev Spider, узятий з офіційного розбиття, і тестова частка, нарізана з таблиці Hugging Face за рядками, є різними об'єктами вимірювання.

### 1.2.2 BIRD

BIRD переносить ту саму міждоменну постановку на великі бази з брудними значеннями [5]. У наборі 12 751 пара на 95 базах у 37 професійних доменах; сумарний обсяг баз близько 33 ГБ. Питання і SQL анотують різні люди, SQL проходить подвійне сліпе анотування і експертний перегляд.

Окремо від Spider BIRD стоїть через поле `evidence`. Це коротка підказка анотації: яке значення якої колонки відповідає словам питання. Питання часто не розв'язується з самих імен таблиць і колонок, бо в тексті є зовнішнє знання і реалістичні значення. Поле `evidence` є частиною пункту набору. Діалоговий контекст попередніх реплік, як у SParC [23] і CoSQL [6], до цього поля не входить: BIRD лишається набором окремих питань.

Друга відмінність — обсяг баз. Поряд зі збігом результату дивляться ефективність виконання, тож головні метрики BIRD — EX і VES [5]. Spider для подальшого експерименту адаптації лишається набором із компактними базами і розбиттям за схемами; BIRD у цьому розділі потрібен як еталон, де золота пара несе ще й підказку `evidence`.

### 1.2.3 Синтетичні корпуси Gretel і OmniSQL

Ручна анотація Spider і BIRD обмежена в обсязі, тому для донавчання збирають синтетичні корпуси. Gretel-Synth містить понад 100 000 прикладів у ділових доменах і поширюється як набір Hugging Face [24]. OmniSQL доводить масштаб далі: понад 2,5 млн екземплярів на більш ніж 16 000 синтетичних базах, із ланцюжком міркувань біля запиту [21].

Такий корпус є навчальним матеріалом. Авторського розбиття, на якому різні системи звітують exact match і execution accuracy, у нього немає. Зростання обсягу саме по собі якість пар не гарантує. У дослідженні Arctic-Text2SQL додавання нижчої за якістю синтетики, зокрема Gretel-Synth, знижувало точність, попри більший train [25]. Для донавчання істотним є вміст пар, а не лише їхня кількість.

## 1.3 Методи адаптації великих мовних моделей до задачі Text-to-SQL

Мовну модель, уже навчену на широкому корпусі, підводять до Text-to-SQL двома режимами: без оновлення параметрів і з їхнім оновленням [1, 3, 19]. Далі розібрано повне донавчання, навчання з кількох прикладів і параметри-

чно ефективну адаптацію, на якій стоїть залежність якості моделі від якості навчальних пар.

### 1.3.1 Повне донавчання і навчання з кількох прикладів

Повне донавчання оновлює всі параметри моделі на парах «питання і схема — SQL». Функція втрат — перехресна ентропія наступного токена, порахована на токенах золотого SQL; токени питання і схеми в втрату не входять. Режим потребує великого train і пам'яті на всі ваги та стани оптимізатора. Для моделі з мільярдами параметрів повне оновлення є дорогим навіть тоді, коли змінюється лише вузька задача.

Навчання з кількох прикладів (few-shot, in-context learning) параметрів не змінює. У контекст кладуть схему, саме питання і кілька золотих пар як зразок [1, 19]. Застосування швидке, окремий цикл навчання відсутній. Обмеженням є довжина контексту: повна схема великої бази і кілька довгих запитів можуть у неї не ввійти. Ваги лишаються тими, що були до прикладу, тож дефект навчального корпусу в параметри моделі цим режимом не записується. Він впливає лише на ті пари, які потрапили в конкретний контекст.

Для задачі, де треба виміряти вплив корпусу на саму модель, потрібен режим, у якому навчається невелика добавка до фіксованих ваг. Його задають LoRA і QLoRA.

### 1.3.2 LoRA і QLoRA

LoRA лишає матрицю ваг попередньо навченої моделі фіксованою і навчає добавку низького рангу [20]. Для лінійної проєкції з входом $x$ вихід має вигляд

$$h = W_0 x + \frac{\alpha}{r} BAx, \tag{1.8}$$

де $W_0$ не оновлюється, $A$ і $B$ — матриці рангу $r$, значно меншого за розміри $W_0$, а $\alpha/r$ — масштабний коефіцієнт. $A$ ініціалізують випадково, $B$ — нулем, тож на початку навчання добавка нульова і вихід збігається з виходом фіксованої проєкції. Градієнт іде в $A$ і $B$. Після навчання добавку можна злити з $W_0$ і отримати одну матрицю без додаткової затримки на виведенні [20].

Рисунок 1.4 показує одну таку проєкцію. У блоці трансформера та сама добавка може стояти на проєкціях уваги $W_Q$, $W_K$, $W_V$, $W_O$ і на трьох проєкціях блока прямого поширення $W_{\text{gate}}$, $W_{\text{up}}$, $W_{\text{down}}$. Hu et al. вводять метод на

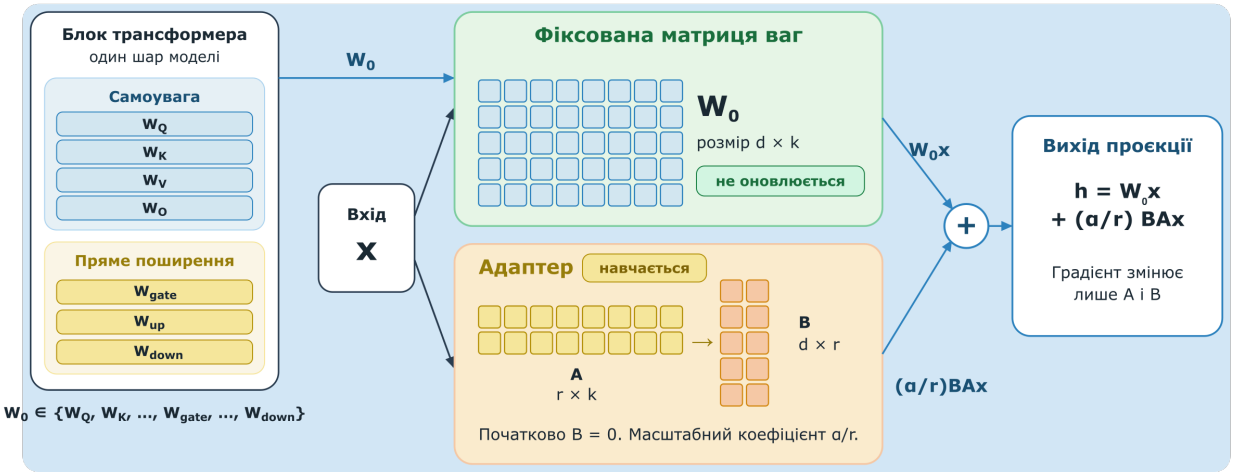



**Рисунок 1.4** – Низькорангова добавка до лінійної проєкції. $W_0$ фіксована; навчаються $A$ і $B$. Початково $B = 0$.

проєкціях уваги [20]; для моделей зі SwiGLU той самий розклад застосовують і до блока прямого поширення. Саме цей набір проєкцій використовує подальший експеримент адаптації.

QLoRA зберігає той самий цикл і ту саму формулу (1.8) [26]. Відмінність у тому, як зберігається $W_0$: її квантують до 4-бітного NormalFloat (NF4), константи квантування квантують ще раз (подвійне квантування), а для множення матрицю деквантують у 16-бітний обчислювальний тип. Градієнт ваг рахують лише для адаптера. Огляд родини LoRA ставить QLoRA серед методів, які зменшують пам'ять [2]. На вимірюваннях Dettmers et al. якість QLoRA тримається на рівні 16-бітного донавчання; виграш полягає в пам'яті, потрібній для адаптації, а не в іншій функції втрат [26]. Окремої схеми обчислення для QLoRA цей розділ не вводить: її задає рисунок 1.4 разом із квантованим зберіганням $W_0$.

### 1.3.3 Залежність якості адаптації від якості навчальних даних

У режимі (1.8) оновлюється невелика добавка, а сигнал для неї дають золоті SQL. Дефект пари входить у втрату нарівні з коректною парою: модель підганяє адаптер до того тексту запиту, який стоїть у train. Тому якість навчальних пар є змінною адаптації, а не лише властивістю еталонної таблиці.

Чутливість Text-to-SQL до вмісту даних зафіксовано й поза донавчанням. Dr.Spider задає 17 типів збурень бази, питання і SQL. На найстійкішій із перевірених моделей відносне падіння усередненої точності виконання становить 14,0 %, а на збуренні еквівалентного вмісту бази — 50,7 % [27]. Це

оцінювання готових систем, і воно показує, що невелика зміна даних змінює виміряну якість.

Для самого навчання той самий висновок дають два незалежні спостереження. QLoRA на невеликому інструкційному наборі високої якості досягає рівня сильніших режимів донавчання [26]. У Text-to-SQL додавання великого, але слабшого синтетичного корпусу здатне погіршити точність [25]. Обсяг train і придатність пар до навчання розходяться.

Окремо показано, що помилки золотих пар змінюють порівняння систем навіть без донавчання. Jin et al. оцінюють частку помилкових анотацій у 52,8 % на BIRD Mini-Dev і 62,8 % на Spider 2.0-Snow. Після виправлення підмножини зі 100 пунктів BIRD Dev вони наново міряють 16 відкритих агентів: відносна зміна якості лежить у межах від $-7$ до $+31$ %, зсув місця в рейтингу — від $-9$ до $+9$, а кореляція Спірмена між початковим і виправленим рейтингами падає з 0,85 до 0,32 [28]. Автори переоцінюють готові агенти і ваги моделей не оновлюють. Питання про те, як виправлення train змінює QLoRA-адаптацію, цим виміром не закрите. Воно потребує засобів, які спочатку відокремлюють різні дефекти пар.

## 1.4 Контроль якості анотацій і наявні засоби аудиту

Дефект золотої пари буває різним за природою. Запит може бути синтаксично прийнятним і водночас містити відомий антипатерн SQL. Він може виконуватися і при цьому не відповідати питанню. Схема або вміст бази можуть бути неузгоджені самі по собі, ще до тексту питання. Далі засоби розведено за цими трьома предметами перевірки.

### 1.4.1 Виявлення антипатернів SQL

Каталог повторюваних хиб SQL і проєктування баз зібрано в Karwin [29]: запит є припустимим для рушія і водночас ненадійний за сенсом або за поведінкою на крайових даних. SQLCheck переносить частину цього каталогу в автоматичні правила і діагностує антипатерн окремого запиту [30].

Для аудиту наборів Text-to-SQL цього замало з двох причин. SQLCheck працює з одним запитом, а не з корпусом золотих пар Spider, BIRD чи синтетичного набору, і не веде діалект, у якому ці пари написано. Покриття правил також вужче за простір, істотний для таких корпусів: поза ним лишаються небезпечні запити зміни даних, порядкозалежна пагінація і шаблони підзапитів,

чутливі до `NULL` [29, 30].

Отже, відкритим лишається діалектне виявлення антипатернів у золотих запитах Spider, BIRD і синтетичних корпусів, разом із класами, які SQLCheck не бере. Цей предмет належить розділу 2.

### 1.4.2 Перевірка семантичної відповідності питання і SQL

Профілювання корпусу описує набір у цілому і не вирішує, чи узгоджена конкретна золота пара. Mitsopoulou і Koutrika зіставляють набори за структурою SQL, властивостями схеми і лексикою питань і розкладають помилки моделі на структурні, операторні й змінні [31]. Процедура придатна для порівняння корпусів і розбору помилок систем. Вердикту про окрему анотацію вона не виносить.

Перевірку окремих пар ведуть кількома лініями, і в автоматичній частині вони спираються на мовну модель. NL2SQL-BUGs дає 2 018 експертно розмічених перекладів, із них 999 із семантичними помилками, і дворівневу таксономію таких помилок. Окремо автори просіюють Spider і BIRD моделлю GPT-4o і підтверджують знайдені помилки вручну; виконуваного конвеєра на весь корпус у випуску немає [32]. Jin et al. будують агента, який кілька кроків працює з базою і складає діагностичний звіт; остаточне рішення лишається за експертом, після чого анотацію виправляють [28]. SQLDriller породжує альтернативні SQL, шукає бази-контрприклади і порівнює результат виконання з тим, який модель передбачає за питанням; за розбіжністю пару позначають і виправляють, а виправлений train Spider і BIRD використовують для донавчання [33]. Спільним для цих ліній є те, що семантичне рішення або його частина залежить від моделі, від людини або від обох.

Детерміновані правила в розглянутих роботах розв’язують сусідню задачу. BLEC зіставляє ключові слова згенерованого тексту з семантичним розбором і зводить три типи правил — заперечення, оператори, словник набору — до бінарної мітки корпусу [34]. Нерозв’язного випадку, коли правилу бракує доказу, метод не має. PV-SQL витягає з питання і необов’язкового `evidence` десять типів обмежень і перевіряє, чи містить згенерований SQL потрібні конструкції [35]. Перевірка вбудована в цикл агента і скеровує виправлення під час виведення. Об’єктом є згенерований запит, а не збережена золота пара. Розбіжність автори відносять до витягнутого обмеження. Обме-

ження поза набором шаблонів опускають, окремого нерозв'язного статусу вони не отримують.

Отже, відкритою лишається детермінована перевірка збережених пар «питання — золотий SQL» на масштабі корпусу: кожна знахідка має спиратися на явний доказ із питання або зі запиту, брак доказу має давати окремий нерозв'язний випадок, а повторюваність шаблону в корпусі не є доказом його правильності. Цей предмет належить розділу 3. Комітет мовних моделей як ширша семантична перевірка розглянуто в наступному пункті; його виміри до цього розділу не входять.

### 1.4.3 Перевірка схеми, даних і оцінювання комітетом мовних моделей

Схема і вміст бази є окремим предметом, бо пара може бути внутрішньо узгодженою, а база, на якій її виконують, — ні. Реляційне профілювання виявляє залежності включення, унікальні комбінації колонок і кандидати у зовнішні ключі [36]. Для наборів на SQLite істотною є поведінка зовнішнього ключа за вимкненого примусового контролю і за семантики `NULL`: порушення посилань тоді не зупиняє запис рядка [37]. Саме так зібрано бази Spider 1.0. SchemaCrawler додає огляд схеми і перевірки на зразок статичного аналізу [38]. Ці засоби дивляться на схему і на відношення як на самостійні артефакти. Знахідка не прив'язана до пункту «питання — SQL».

Виконання запиту на базі є третьою, вже динамічною перевіркою: запит може розібратися і при цьому впасти на даних або повернути результат, який не випливає з питання. Синтаксична прийнятність, антипатерн, семантика пари і виконуваність на конкретній базі дають різні списки дефектів. Злиття цих списків в одну позначку «пара погана» стирає, який саме дефект виправлено.

Там, де правило не має доказу, семантику оцінюють комітетом мовних моделей. Сильний суддя наближає людські переваги і водночас має зміщення позиції, багатослівності, самопідсилення і слабку ділянку міркувань [39]. Комітет і обмін аргументами між суддями зменшують залежність від однієї моделі і виводять розбіжність назовні [40]. Для аудиту Text-to-SQL звідси випливає режим користування, а не заміна еталона: контрольована підказка, кілька моделей, рішення більшістю і подальше калібрування людиною. Вердикт комітету є сигналом, який треба підтвердити, перш ніж правити золоту пару.

Таблиця 1.1 зводить засоби цього підрозділу і двох попередніх за предметом перевірки і за тим, чого бракує для аудиту корпусу золотих пар.

**Таблиця 1.1** – Засоби контролю якості анотацій Text-to-SQL

| Засіб | Що перевіряє | Обмеження для аудиту корпусу |
|---|---|---|
| Mitsopoulou, Koutrika [31] | структуру SQL, схему, лексику питань | немає вердикту про окрему золоту пару |
| SQLCheck [30] | антипатерни одного запиту | не корпус і не діалект; частина класів поза правилами |
| BLEC [34] | ключові слова тексту проти розбору | бінарна мітка; нерозв'язного випадку немає |
| PV-SQL [35] | обмеження в згенерованому SQL | золоту анотацію не аудитує |
| NL2SQL-BUGs [32] | семантичні помилки перекладу | немає конвеєра на весь корпус |
| Jin et al. [28] | семантику анотації агентом | рішення моделі і людини; донавчання немає |
| SQLDriller [33] | узгодженість за виконанням і виправлення | донавчання після ремонту без поділу груп правок |
| Профілювання схеми [36–38] | ключі, залежності, цілісність | знахідка не прив'язана до пари питання — SQL |

Jin et al. після виправлення наново міряють готових агентів і ваги не оновлюють [28]. SQLDriller уже донавчає модель на виправленому train [33]. В обох випадках виправлення злито: з результату не видно, який внесок дають окремо антипатерни, семантичні суперечності, підтверджені знахідки комітету і дефекти схеми та даних.

Отже, відкритим лишається багатовимірний аудит, у якому ці чотири джерела правок зберігають окремі списки, і вимірювання того, як кожне джерело змінює QLoRA-адаптацію на авторському розбитті Spider. Цей предмет належить розділу 4.

### Висновки до розділу 1

Огляд фіксує задачу як перехід від питання і схеми до SQL, а якість розв'язку — як повний збіг форми і як збіг результату виконання. Еталонні набори WikiSQL, Spider і BIRD дають золоті пари й авторське розбиття, на якому ці метрики порівнянні. BIRD додає поле `evidence` і ефективність виконання на великих базах. Gretel і OmniSQL є навчальними корпусами: масштаб у них великий, а лідербордної ролі немає. Адаптацію, у якій дефект пари здатний увійти в параметри, задає QLoRA: фіксована матриця

зберігається квантованою, навчається низькорангова добавка.

Зіставлення засобів контролю залишає три прогалини.

Діалектного виявлення антипатернів у золотих запитах Spider, BIRD і синтетичних корпусів, із класами поза правилами SQLCheck, наявні засоби не дають. Цю прогалину закриває розділ 2.

Детермінованої перевірки збережених пар «питання — золотий SQL», із явним доказом і окремим нерозв'язним випадком, розглянуті методи не дають: або рішення залежить від моделі, або правило обслуговує генерацію і не аудитує золоту анотацію. Цю прогалину закриває розділ 3.

Вплив чотирьох окремих груп виправлень на QLoRA-адаптацію не виміряно. Виправлення анотацій або змінює рейтинг готових агентів без донавчання, або входить у донавчання вже злитим корпусом. Цю прогалину закриває розділ 4.

**Список використаних джерел до розділу 1**


[1] Liang Shi, Zhengju Tang, Nan Zhang, Xiaotong Zhang, and Zhi Yang. A survey on employing large language models for Text-to-SQL tasks. *ACM Computing Surveys*, 58(2):1–37, 2025. doi: 10.1145/3737873.

[2] Yuren Mao, Yuhang Ge, Yijiang Fan, Wenyi Xu, Yu Mi, Zhonghao Hu, and Yunjun Gao. A survey on LoRA of large language models. *Frontiers of Computer Science*, 19:197605, 2025. doi: 10.1007/s11704-024-40663-9.

[3] Ali Mohammadjafari, Anthony S. Maida, and Raju Gottumukkala. From natural language to SQL: Review of LLM-based Text-to-SQL systems, 2024.

[4] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and Text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921. Association for Computational Linguistics, 2018. doi: 10.18653/v1/D18-1425.

[5] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma,

Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM already serve as a database interface? A big bench for large-scale database grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, pages 42330–42357, 2023.

[6] Tao Yu, Rui Zhang, Heyang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, Youxuan Jiang, Michihiro Yasunaga, Sungrok Shim, Tao Chen, Alexander Fabbri, Zifan Li, Luyao Chen, Yuwen Zhang, Shreya Dixit, Vincent Zhang, Caiming Xiong, Richard Socher, Walter Lasecki, and Dragomir Radev. CoSQL: A conversational Text-to-SQL challenge towards cross-domain natural language interfaces to databases. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 1962–1979, 2019. doi: 10.18653/v1/D19-1204.

[7] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. A survey of Text-to-SQL in the era of LLMs: Where are we, and where are we going? *IEEE Transactions on Knowledge and Data Engineering*, 37(10):5735–5754, 2025. doi: 10.1109/TKDE.2025.3592032.

[8] W. A. Woods, R. M. Kaplan, and B. Nash-Webber. The lunar sciences natural language information system: Final report. Technical Report 2378, Bolt Beranek and Newman Inc., June 1972. URL https://brenocon.com/Woods1972Lunar.pdf.

[9] Yunyao Li, Huahai Yang, and H. V. Jagadish. NaLIX: An interactive natural language interface for querying XML. In *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, 2005. URL https://yunyaoli.github.io/files/sigmod05-nalix.pdf.

[10] Fei Li and H. V. Jagadish. Constructing an interactive natural language interface for relational databases. *Proceedings of the VLDB Endowment*, 8 (1):73–84, 2014. URL https://www.vldb.org/pvldb/vol8/p73-li.pdf.

[11] Li Dong and Mirella Lapata. Language to logical form with neural

attention. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 33–43, 2016. doi: 10.18653/v1/P16-1004.

[12] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2SQL: Generating structured queries from natural language using reinforcement learning. *CoRR*, abs/1709.00103, 2017. doi: 10.48550/arXiv.1709.00103.

[13] Xiaojun Xu, Chang Liu, and Dawn Song. SQLNet: Generating structured queries from natural language without reinforcement learning, 2017.

[14] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. RAT-SQL: Relation-aware schema encoding and linking for Text-to-SQL parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7567–7578, 2020. doi: 10.18653/v1/2020.acl-main.677.

[15] Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. TaBERT: Pretraining for joint understanding of textual and tabular data. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8413–8426, 2020. doi: 10.18653/v1/2020.acl-main.745.

[16] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901, 2021. doi: 10.18653/v1/2021.emnlp-main.779.

[17] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. Evaluating the Text-to-SQL capabilities of large language models, 2022.

[18] Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of Text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems 36*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/72223cc66f63ca1aa59edaec1b3670e6-Abstract.html.

[19] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment*, 17(5):1132–1145, 2024. doi: 10.14778/3641204.3641221.

[20] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of the 10th International Conference on Learning Representations*, 2022.

[21] Haoyang Li, Shang Wu, Xiaokang Zhang, Xinmei Huang, Jing Zhang, Fuxin Jiang, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, Hong Chen, and Cuiping Li. OmniSQL: Synthesizing high-quality Text-to-SQL data at scale. *Proceedings of the VLDB Endowment*, 18(11):4695–4709, 2025. doi: 10.14778/3749646.3749723.

[22] Catherine Finegan-Dollak, Jonathan K. Kummerfeld, Li Zhang, Karthik Ramanathan, Sesh Sadasivam, Rui Zhang, and Dragomir Radev. Improving Text-to-SQL evaluation methodology. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 351–360, 2018. doi: 10.18653/v1/P18-1033.

[23] Tao Yu, Rui Zhang, Michihiro Yasunaga, Yi Chern Tan, Xi Victoria Lin, Suyi Li, Heyang Er, Irene Li, Bo Pang, Tao Chen, Emily Ji, Shreya Dixit, David Proctor, Sungrok Shim, Jonathan Kraft, Vincent Zhang, Caiming Xiong, Richard Socher, and Dragomir Radev. SParC: Cross-domain semantic parsing in context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4511–4523, 2019. doi: 10.18653/v1/P19-1443.

[24] Gretel.ai. Synthetic Text-to-SQL dataset. Hugging Face Datasets, 2024. URL https://huggingface.co/datasets/gretelai/synthetic_text_to_sql.

[25] Zhewei Yao, Guoheng Sun, Łukasz Borchmann, Zheyu Shen, Minghang Deng, Bohan Zhai, Hao Zhang, Ang Li, and Yuxiong He. Arctic-Text2SQL-R1: Simple rewards, strong reasoning in Text-to-SQL. In *Findings of the*

*Association for Computational Linguistics: ACL 2026*, pages 26966–26995, 2026. doi: 10.18653/v1/2026.findings-acl.1345.

[26] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36*, 2023.

[27] Shuaichen Chang, Jun Wang, Mingwen Dong, Lin Pan, Henghui Zhu, Alexander Hanbo Li, Wuwei Lan, Sheng Zhang, Jiarong Jiang, Joseph Lilien, Steve Ash, William Yang Wang, Zhiguo Wang, Vittorio Castelli, Patrick Ng, and Bing Xiang. Dr.Spider: A diagnostic evaluation benchmark towards Text-to-SQL robustness. In *Proceedings of the 11th International Conference on Learning Representations*, 2023.

[28] Tengjun Jin, Yoojin Choi, Yuxuan Zhu, and Daniel Kang. Pervasive annotation errors break Text-to-SQL benchmarks and leaderboards. *Proceedings of the VLDB Endowment*, 19(5):931–944, 2026. doi: 10.14778/3796195.3796206.

[29] Bill Karwin. *SQL Antipatterns: Avoiding the Pitfalls of Database Programming*. Pragmatic Bookshelf, 2010.

[30] Prashanth Dintyala, Arpit Narechania, and Joy Arulraj. SQLCheck: Automated detection and diagnosis of SQL anti-patterns. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 2331–2345, 2020. doi: 10.1145/3318464.3389754.

[31] Anna Mitsopoulou and Georgia Koutrika. Analysis of Text-to-SQL benchmarks: Limitations, challenges and opportunities. In *Proceedings of the 28th International Conference on Extending Database Technology*, pages 199–212, 2025. doi: 10.48786/EDBT.2025.16.

[32] Xinyu Liu, Shuyu Shen, Boyan Li, Nan Tang, and Yuyu Luo. NL2SQL-BUGs: A benchmark for detecting semantic errors in NL2SQL translation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5662–5673, 2025. doi: 10.1145/3711896.3737427.

[33] Yicun Yang, Zhaoguo Wang, Yu Xia, Zhuoran Wei, Haoran Ding, Ruzica Piskac, Haibo Chen, and Jinyang Li. Automated validating and fixing of Text-to-SQL translation with execution consistency. *Proceedings of the ACM on Management of Data*, 3(3):1–28, 2025. doi: 10.1145/3725271.

[34] Chang Shu, Yusen Zhang, Xiangyu Dong, Peng Shi, Tao Yu, and Rui Zhang. Logic-consistency text generation from semantic parses. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 4414–4426, 2021. doi: 10.18653/v1/2021.findings-acl.388.

[35] Yuan Tian and Tian Zhang. PV-SQL: Synergizing database probing and rule-based verification for Text-to-SQL agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 25827–25845, 2026. doi: 10.18653/v1/2026.findings-acl.1286.

[36] Ziawasch Abedjan, Lukasz Golab, and Felix Naumann. Profiling relational data: A survey. *The VLDB Journal*, 24(4):557–581, 2015. doi: 10.1007/s00778-015-0389-y.

[37] Mozhgan Memari, Sebastian Link, and Gillian Dobbie. SQL data profiling of foreign keys. In *Proceedings of the 34th International Conference on Conceptual Modeling*, pages 229–243, 2015. doi: 10.1007/978-3-319-25264-3_17.

[38] Sualeh Fatehi. SchemaCrawler: Database schema discovery and comprehension tool. https://www.schemacrawler.com/, 2026.

[39] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36*, 2023.

[40] Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. ChatEval: Towards better LLM-based evaluators through multi-agent debate. In *Proceedings of the 12th International Conference on Learning Representations*, 2024.